

Hackathon Technical Solution Report

Field	Details
Problem ID	P5
Problem Title	Intelligent Fine-Grained Vehicle Recognition and Analysis System
Team Name	T50 "Clover"
Team Members	Priyanshu Kumar Senapati (1RUA24CSE0337), Saatwik Kumar Yadav (1RUA24CSE0390), Rachana Bhaskar Gowda (1RUA24CSE0348), R Muni Disnitha (1RUA24CSE0345)
GitHub Repository	https://github.com/Priyanshu-senapati/Rescuelens (commit `eb587b6`, the code and results this report describes)
Live Demo / Deployment	https://priyanshu-senapati--rescuelens-web.us-east.modal.run

Work Division

Member	Owns
Priyanshu Kumar Senapati	Classifier training, calibration, safety metrics, deployment
Saatwik Kumar Yadav	Product and UI: live camera, crash handling, browser tests, CI
Rachana Bhaskar Gowda	Safety content and real-world validation
R Muni Disnitha	Evidence, documentation and demo

AI Assistance Declaration

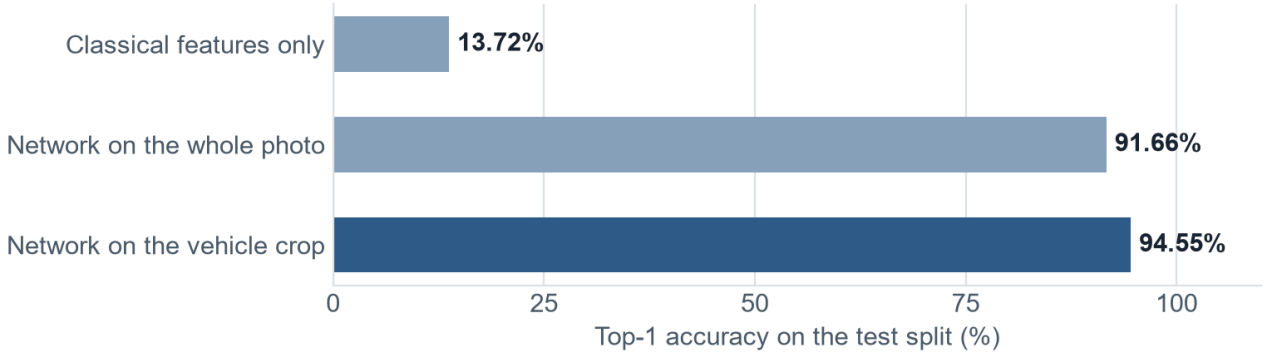
Tool	Used by	Used for
Claude Code (Anthropic)	Priyanshu Kumar Senapati	Classifier training, calibration, safety metrics and deployment
Claude Code (Anthropic)	Saatwik Kumar Yadav	Live camera, rescue-sheet drawing, stress sweep, crash screen, offline mode, browser tests and CI; finding and reading the manufacturer guides and statistics; README, viva sheets and charts .

1. Problem Understanding & Solution Strategy

- **Core Technical Challenge:** Stanford Cars asks for one label out of 196, and many of those labels are nearly the same car. An Audi S5 and an Audi A5 differ mainly in the grille, bumpers and badges, and that pair was our most frequent test error (15 photos). A Hyundai Sonata and a Sonata Hybrid share a body. Real photos then add darkness, smoke, blur, low resolution, occlusion and awkward angles.

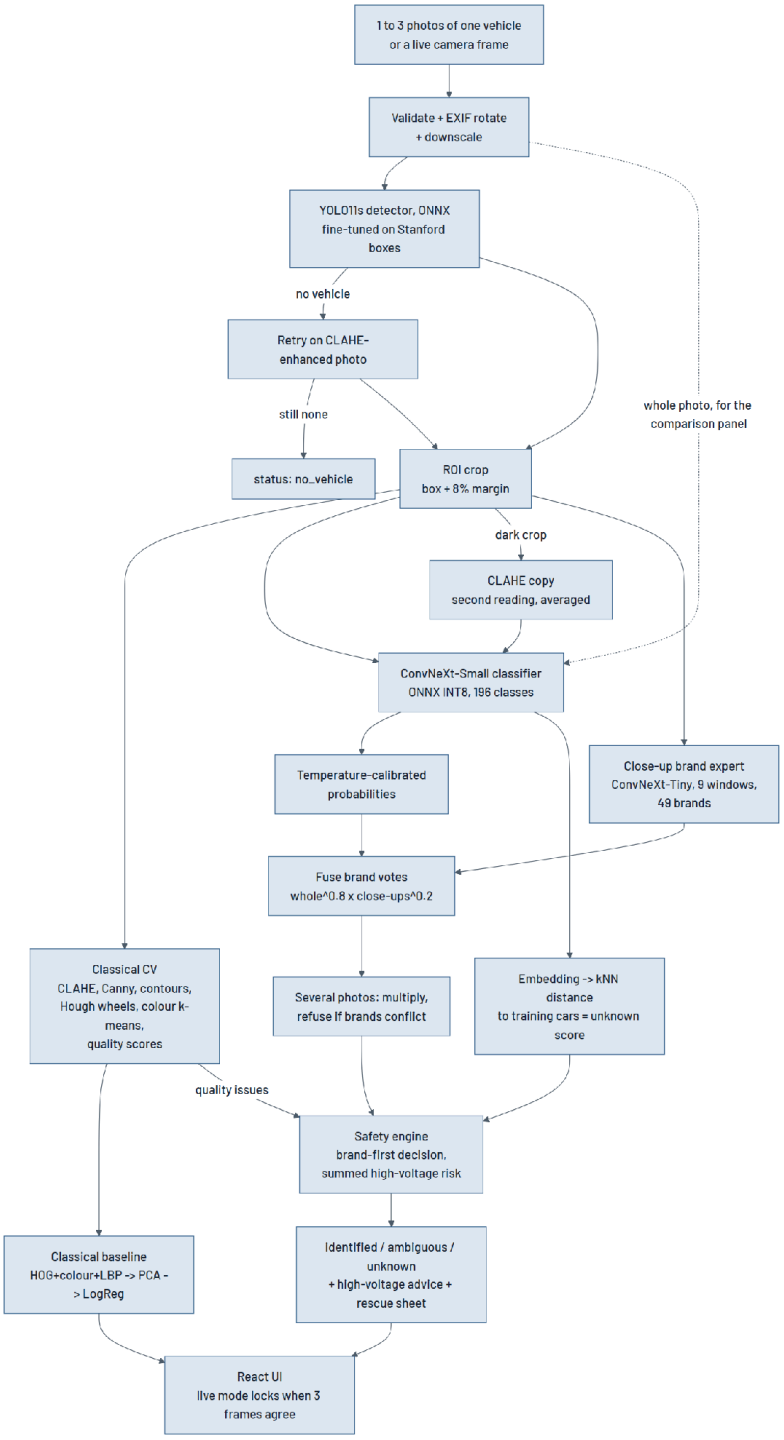
We designed this system for the case of crash rescue, wherein there is actually an expense for an incorrect prediction. The rescuers have to know the type of vehicle before any cuts are made because hybrids and electric vehicles have high voltage batteries and wiring, which are not visible on the exterior, and in the event of a crash, the plate or VIN may also be crushed or concealed. Thus, we had two types of failures to guard against: failure to recognize the hybrid, and making a confident prediction for a vehicle which the model has not seen, which is quite likely since the training data goes only up to 2012 for US vehicles.

- **Proposed Methodology:** The YOLO11s will detect the car, classical computer vision will detect the confidence of the image, and the ConvNeXt-Small will classify the crop. After that, a safety mechanism will map the calibrated probabilities to one of the three decisions (detected, no confidence, not detected) and warn about high voltage if any of the nine classes of hybrids or electric vehicles together is above 5%. This was preferred over a static baseline since the classical features alone gave us a top-1 performance of 13.72%, the network on the entire image gave us a 91.66% accuracy, and the top answer alone would have missed 4.32% of the vehicles compared to 1.15%.



2. Technical Architecture & Implementation

The code is divided according to the job: backend/app/classical.py (classical CV), models.py (ONNX models), safety.py (decision rules), pipeline.py (orchestration), main.py (API and validation). Training code is under training/ and React interface is under frontend/. All rationale for design decisions are in docs/design.md.



2.1 Classical Algorithmic Baseline

- **Algorithms Utilized:**

P5 requirement	What we used
Preprocessing and normalization	EXIF rotation, downscaling to 1600 px, CLAHE on the LAB lightness channel, ImageNet normalization
Edges and contours	Bilateral filter, Canny, morphological closing, largest outer contour as the body outline
Colour and texture	HSV histogram, k-means dominant colours on the central 70% of the crop, 8-neighbour LBP histogram
Region of interest	Detector box plus an 8% margin, the same rule used in training

Geometry	Aspect ratio, contour extent and solidity, Hough circles for wheels and a wheelbase ratio

For the required comparison we also trained a classical-only classifier: HOG, HSV colour, LBP and aspect ratio, reduced with PCA and passed to logistic regression. It reached 13.72% top-1 and 34.03% top-5.

- **Adaptive Selection Logic:** Thresholds are derived from the picture itself using values that are 0.66 and 1.33 times the median gray level of the picture. Hough transform search is done relative to the car: it is performed within the bottom 60% of the picture and for radius sizes in range 8-30% of the height of the crop. CLAHE settings are fixed (clip limit of 2.0, 8x8 tile grid), but the application of CLAHE is dependent on the input data. In case no cars are detected in the first attempt, the program tries again and gives another chance to the classifier if the brightness of the crop is less than 30/100. Tests of brightness, contrast, Laplacian, resolution, and cutoff give simple explanations such as "Low light" or "Blurry".

2.2 Deep Learning Pipeline

- **Model Architecture:**

Model	Role	Why
ConvNeXt-Small, 384 px, 49.6 M parameters	196-class classifier	ImageNet-22k weights transfer well to fine-grained work. We trained ConvNeXt-Tiny first, then moved up: validation top-1 rose from 93.74% to 94.23%, and unseen cars caught rose from 88.3% to 96.7%
YOLO11s, 640 px	Vehicle detector	Started from COCO weights, then fine-tuned for 20 epochs on the Stanford train boxes as one class. Mean IoU rose from 0.886 to 0.954
ConvNeXt-Tiny, 224 px windows	Close-up brand expert	Badges are a few pixels wide in a 384 px view of a whole car, so this model votes on 49 brands over 9 zoomed windows

- **Loss Functions & Training:** The classifier applies cross entropy loss function with label smoothing of 0.1 and AdamW optimizer (learning rate 2e-4 and weight decay 0.05), one epoch warm-up, cosine decay schedule, mixed precision training and stochastic depth of 0.1. Data augmentation includes random resized crop, horizontal flip, color jitter, random darkening that simulates nighttime and random erasing that simulates occlusion. The data set was Stanford Cars: official training images were split in 90% and 10% with stratification and seed 42, the best validation epoch was selected and the validation used 8,041 test images only. Brand expert uses the same loss function with AdamW optimizer while the detector applies default Ultralytics loss functions. Temperature scaling on validation ($T = 0.768$) improved calibration error (ECE) from 0.098 to 0.011. For model deployment, we use ONNX format and apply dynamic INT8 quantization to MatMul and Gemm layers because ConvNeXt has many linear layers and ConvInteger operation is not supported by ONNX Runtime CPU provider for int8.

2.3 Hybrid Integration & Adaptive Mechanism

- **Fusion Strategy:** Some connections between the classical and deep sub-models exist. Detection defines which crop the classifier gets. CLAHE provides detection with a second chance in the dark scene and a second chance for the classifier to estimate the dark crops, averaging the results. Quality measures justify the low confidence to the user. In addition, we have tested late fusion of classical

and deep probabilities, and the optimal weight was 0 on validation, hence there was nothing new provided by the classical model, and it was omitted from classification. The brand expert fuses the whole car brand scores using a product rule (whole^0.8 x close-ups^0.2), determined on validation. One can also upload up to 3 images of the same vehicle: probabilities multiply, and different brands are never fused if the photo is confident enough.

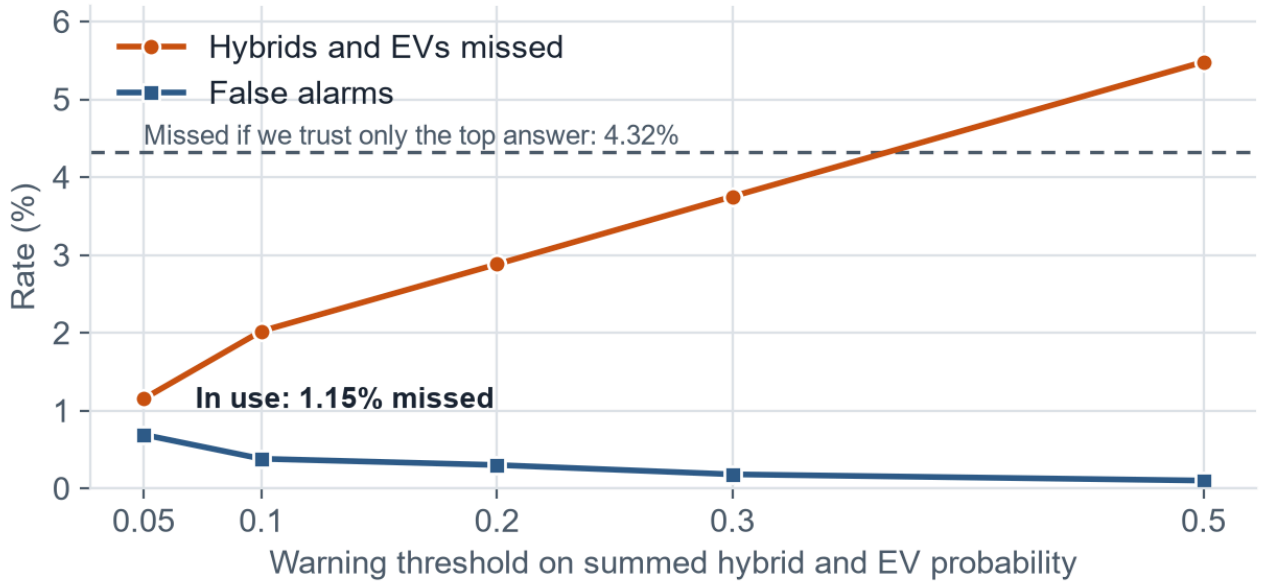
- **Decision Logic:** The template's example is about balancing amplification against noise. For us the equivalent question is how much to conclude from uncertain evidence:

  1. The brand with the most summed probability wins, then the strongest family inside it, then the likeliest model, so a lone top guess can't overrule the combined evidence for another brand.

  2. The answer stops at the deepest level it is sure of: a model at 0.6 or above, then a family at 0.6 or above, and otherwise the brand alone.

  3. A car is called unknown when its brand probability is below 0.5, or when its embedding is far from the training cars. That distance is 1 minus the mean cosine similarity to the 5 nearest training embeddings, with the threshold at the 95th percentile of validation scores (0.369).

  4. The high-voltage warning uses summed probability with a threshold of 0.05, the lowest we tested, because a missed high-voltage car can kill and a false alarm only costs time. Unknown cars are treated as possibly high voltage. When a hybrid or EV is likely, the app shows where its battery and cables are, taken from the manufacturer's guide for all 9 models (`docs/hazard-sources.md`).



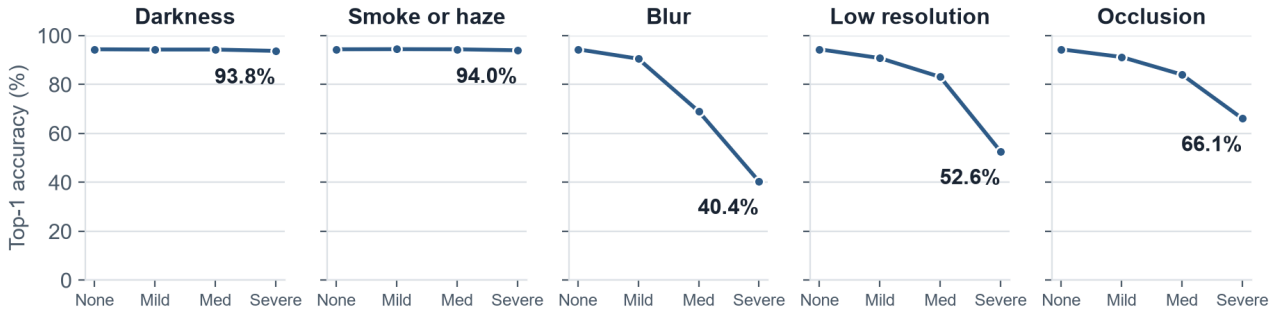
3. Experimental Benchmarks & Evaluation

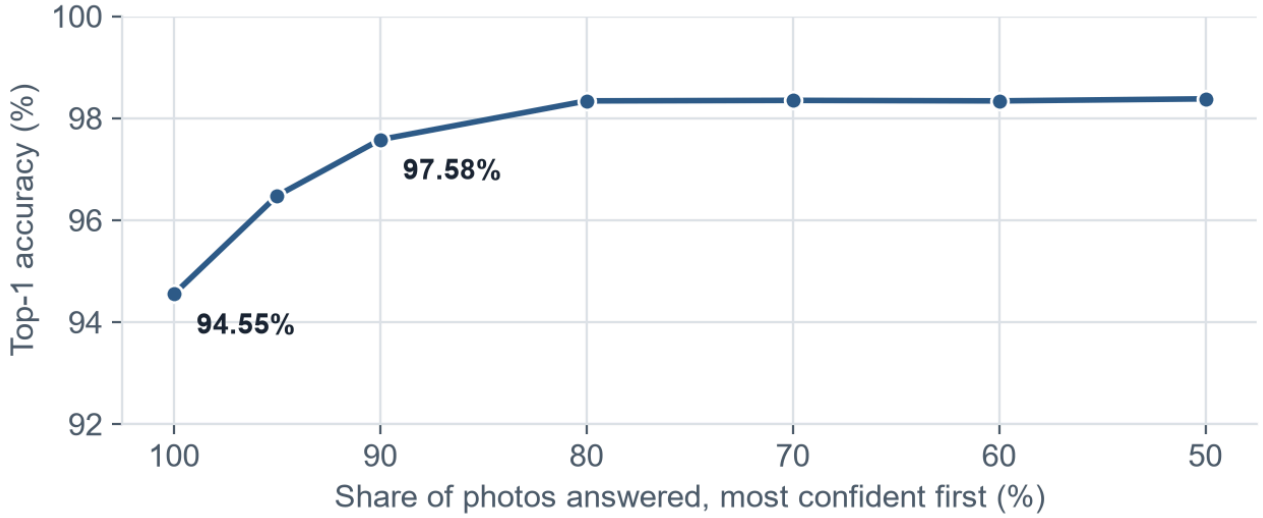
3.1 Quantitative Benchmarks

The template's PSNR, SSIM and LPIPS columns measure image enhancement and don't apply to P5, so we report the metrics P5 asks for. All numbers come from `eval_P5_T50.md` unless stated otherwise. The scripts in `training/` and `tools/` produce them, and the README lists the steps. The dataset, model files and the test photos for the unseen-car and crash rows aren't stored in git, so they have to be supplied to rerun those tests.

Approach / Model Variant	Top-1 ↑	Top-5 ↑	Mean IoU ↑	Latency
Classical Baseline	13.72%	34.03%	0.639 (edge contours)	Not timed separately
Deep Learning Only (whole photo)	91.66%	Not measured	No localization	302 ms (classifier, 2 CPU threads)
Proposed Hybrid System	94.55%	99.27%	0.954	About 1 to 2 s (full answer, live site)

P5 metric	Result
Classification	Top-1 94.55%, top-5 99.27%, 94.70% with flip TTA
Object localization	Mean IoU 0.954, with 99.85% of boxes at IoU 0.5 or above (COCO weights: 0.886 and 92.81%)
Object detection	mAP@0.5 0.999, mAP@0.5:0.95 0.941 (COCO weights: 0.892 and 0.825)
Model efficiency	Classifier 189.4 MB to 54.3 MB, 739 ms to 302 ms on 2 CPU threads, accuracy unchanged at 93.67% on a 300-image check
Confidence	ECE 0.098 to 0.011. Most confident 90% of photos: 97.58% accurate
Missed hybrids and EVs (347 photos)	1.15% with the summed rule, 4.32% trusting the top answer, 0.69% false alarms
Cars outside the 196 classes (60 photos, 30 models)	96.67% answered unknown, while 8.33% of known cars were wrongly flagged
Brand accuracy, 49 brands	98.61% on clean photos, 92.05% with 30% of the vehicle hidden
Real crash photos (48)	44 declined. Of 3 confident answers, 1 was wrong





- **Analysis:** The crop helped accuracy the most. Quantization helped speed the most, and we measured no accuracy loss from it on our 300-image check. Darkness and haze barely hurt because training already darkens images, while blur and low resolution remove the fine detail this task

depends on. The summed rule trades a small rise in false alarms for far fewer missed hybrids. With all the models running, a full answer on the live site takes about 1 to 2 s, well inside the 10-second limit.

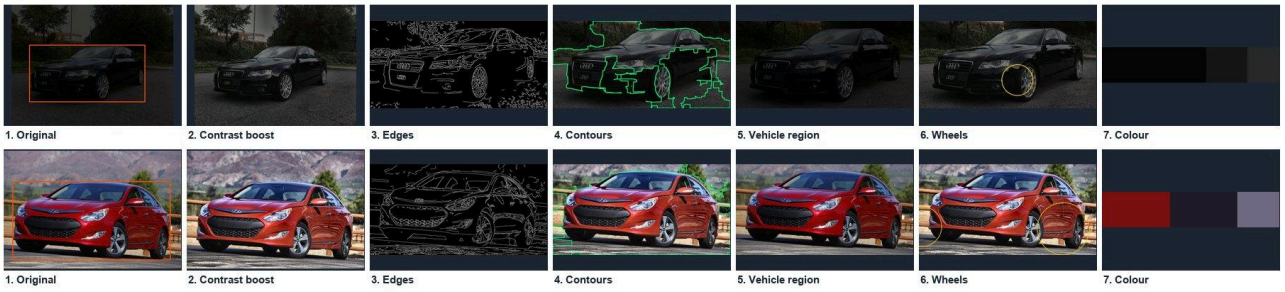
The number of 94.55% was obtained on the crops extracted from the annotated boxes in the dataset using only the classifier. This doesn't account for any errors by our detector, nor does it incorporate brand expert detection and night readings which were done separately. Given the fact that there is 99.85% of boxes by the detector at IoU equal or greater than 0.5, we can assume that the pipeline will perform similarly.

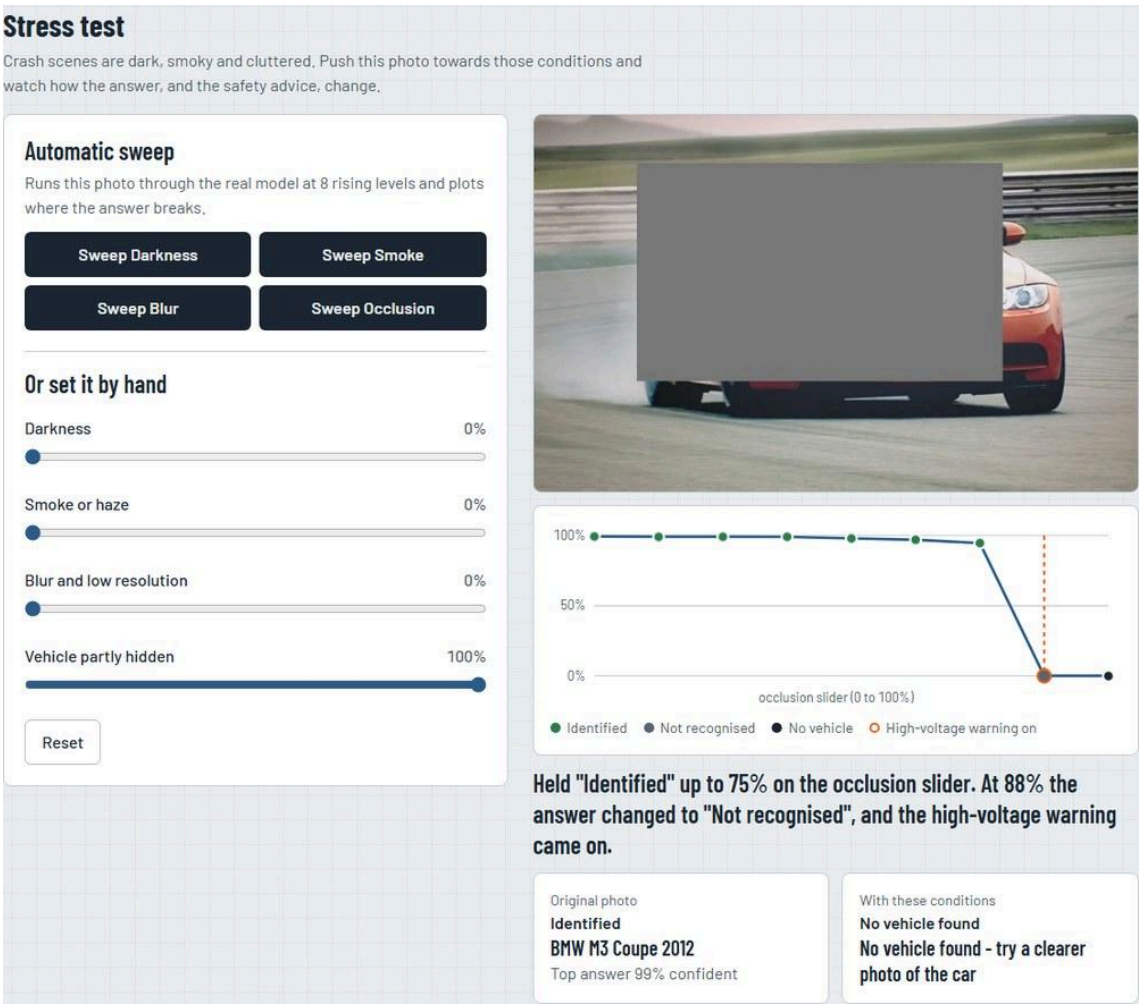
3.2 Visual Outputs & Intermediate Representations

1. **Extreme low-light:**As shown in the image below, for step 1 in the first row, there is a dark image, while for step 2, there is a CLAHE processed image. The live object still recognizes this vehicle to be an Audi S4 Sedan 2012 with 91% accuracy.

2. **Non-uniform / backlit scenes:**Backlit images were not separately tested as a scenario in this research. CLAHE algorithm operates on local tiles, thus preventing any excessive exposure of bright areas. Partial occlusion would be the nearest scenario, in which, for example, a BMW can identify "Identified" state up to 75% on occlusion bar, but then changes it to "Not recognised."

3. **Intermediate states:** Each classical step is outputted as an image and presented within the application: original with a box, contrast, edges, contour, vehicle region, wheels and color.





The three correct outputs from the ground truth model are a Tesla Model S classified based on battery position, a Chevrolet Silverado with hidden rear classified as "not sure", and a Tata Nexon that is not in the database classified as "not recognized".



3.3 Ablation Study

Variant	Top-1	What it shows
Full system	94.55%	Reference
Without classical fusion	94.55%	No change. The fusion weight was 0 on validation
Without the crop (whole photo)	91.66%	The crop adds 2.89 points, the biggest single gain
With flip TTA	94.70%	Adds 0.15 points for double the inference time, so we left it out
FP32 instead of INT8 (300 images)	93.67% vs 93.67%	Quantization lost nothing on this subset
CLAHE before the classifier, severe darkness	93.75% to 93.90%	Very little, because training already darkens images

Improvements that are not captured in top-1: Tweaking the detector increased IoU mean to 0.954 from 0.886; the brand expert improved brand accuracy from 98.30% to 98.61% for clean images and 90.75% to 92.05% for 30% occluded ones. The night reading scored neutral to +0.5 for artificially darkened images.

4. Failure Modes & Limitations

- **Observed Failure Case:** The one we did worst at was the Mahindra Thar. Out of 8 images, 5 came back "Identified: Jeep Wrangler" with 65 to 99% accuracy with no high voltage warning. The remaining 3 images returned an answer of "unknown." Other misses include the Audi S5 identified incorrectly as an A5 (15 test photos), the Tahoe Hybrid with upper rear part not visible identified as an Avalanche at 98% confidence with no warning and top-1 drop at 40.40% from severe blur, and

brand correct only 22.92% of the time on 48 actual crash test images, dropping 44 while only making 1 sure misidentification (a Ford identified as a Chevrolet).

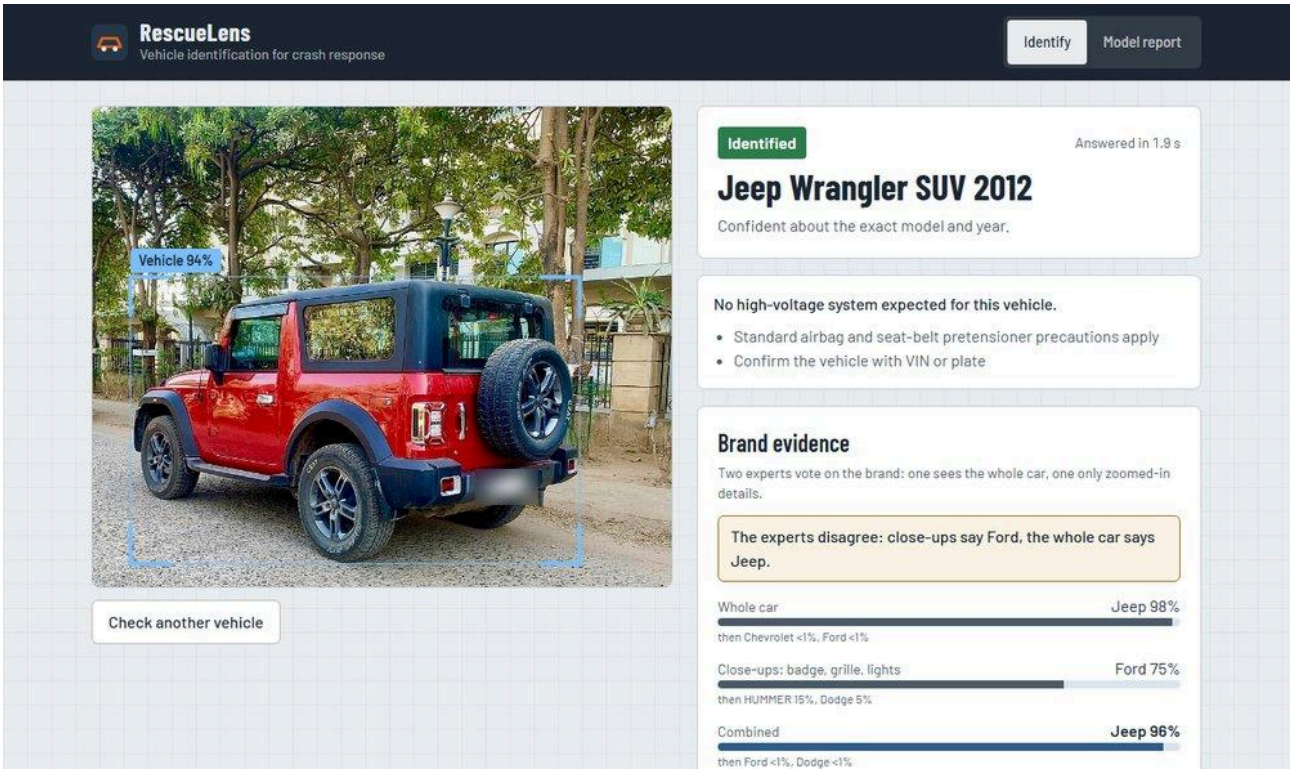


Photo: Mahindra Thar SUV in "Red Rage" colour, by Ank Kumar, CC BY-SA 4.0, Wikimedia Commons. Plate blurred.

- **Root Cause & Mitigation:**

Failure	Root cause	Mitigation
Thar taken for Wrangler	The Thar is styled like the Wrangler, so its embedding sits close to a known class and passes the distance check	The app always says to confirm with VIN or plate. Next: lower the answer when the brand experts disagree (they did on the photo we inspected), and add look-alikes to a held-out set when tuning the threshold
S5 taken for A5	Trims and facelifts change only grilles, bumpers and badges, which are tiny at 384 px	Family-level answers, plus the close-up brand expert
Tahoe taken for Avalanche	The two share a front end, and the detector's box stops at the occluder	The multi-photo option lets a rescuer add another side
Severe blur	Blur removes the high-frequency detail that separates similar cars	The "Blurry" check gives a reason for the low confidence
Real crash photos	Wrecks and unseen models are far from the training data	Collect labelled damaged vehicles, which Stanford Cars doesn't have

Other limits: The training set comprises US cars before 2012; thus, almost all the cars that are currently available on Indian roads would get "unknown" answers. The robustness tests have been performed through simulated degradation. The offline mode has not been used since it has been done using proxy models; therefore, offline mode has been turned off in the production environment. Out of nine guide links, seven are hosted by fire service departments, a rescue training center, or a rescue software provider. None of the responders have checked the safety instructions.

5. Deployment & Quickstart Guide

Quickstart Commands

```
# 1. Clone the repository and install dependencies
git clone https://github.com/Priyanshu-senapati/Rescuelens
cd Rescuelens
pip install -r backend/requirements.txt

# 2. Run inference on a sample directory (needs the model files in ./models)
python tools/check_samples.py --samples ./frontend/public/samples --models ./models

# Or run the whole app in Docker (MODEL_BASE_URL is a folder holding the model files)
docker build -t rescuelens --build-arg MODEL_BASE_URL=<folder-with-the-model-files> .
docker run -p 7860:7860 rescuelens     # open http://localhost:7860
```

Model weights aren't in git because of GitHub's file size limit. Without them the app still runs but serves sample answers and says so on screen, so real answers need the live URL or the model files.

Deployment Format

- A container hosts the React interface, the FastAPI service, and ONNX runtime. It is deployed on Modal using one warm container (2 CPUs, 2GB) from modal_app.py, while another uses a Dockerfile for the deployment of the same application. The GitHub actions test the back end, type-check, build, and browser tests.

Student Submission Checklist

☑ GitHub repository link provided with an updated README.md file.

☑ Code is not directly pasted into this document.

☑ Table data is consistent with reproducible test results from the repository.

☑ Information about all team members provided.